

EMATM0048: Software Development Programming and Algorithms (SDPA)

Tutorial 2

In this tutorial we will:

- String type and methods
- Conditional statements
- Iterations in Python
- Nested loops

1. String type

In Python, A string is a sequence of characters enclosed within a single quote or double quote. These characters could be anything like letters, numbers, or special symbols enclosed within quotation marks.

The string type in Python is represented using a str class.

<https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str>

To work with text or character data in Python, we use Strings. Once a string is created, we can do many operations on it, such as searching inside it, creating a substring from it, and splitting it.

Note: You can use single or double quotes to assign a string. When using single quotes, you will face problems if the value to be assigned itself contains single quotes too. For example to assign the string in these bracket(single quotes are '). Try the following, do you notice the difference?

```
In [ ]: print("single quotes are ' '")
        print('single quotes are ''')
```

String special operators

Operator	Description	Example
+	Concatenation - Adds values on either side of the operator	a + b will give HelloPython
*	Repetition - Creates new strings, concatenating multiple copies of the same string	a*2 will give -HelloHello
[]	Slice - Gives the character from the given index	a[1] will give e
[:]	Range Slice - Gives the characters from the given range	a[1:4] will give ell
in	Membership - Returns true if a character exists in the given string	H in a will give 1
not in	Membership - Returns true if a character does not exist in the given string	M not in a will give 1

Check String

To check if a certain phrase or character is present in a string, we can use the keyword `in`.

```
In [ ]: txt = "The best things in life are free!"
        print("free" in txt)
```

Check if NOT: To check if a certain phrase or character is NOT present in a string, we can use the keyword `not in`.

```
In [ ]: txt = "The best things in life are free!"
        print("expensive" not in txt)
```

Looping Through a String

Since strings are arrays, we can loop through the characters in a string, with a `for` loop.

```
In [ ]: for x in "banana":
        print(x)
```

String Formatting Operator

The `%` symbol has special meaning in Python strings. It can be used as a placeholder for another value to be inserted into the string. The `%` symbol is a prefix to another character which defines the type of value to be inserted.

It works like this:

```
In [ ]: print("Hello %s, you scored %i out of %i" % ("Homer", 3, 100))
```

Common Python String Methods

There are numerous methods available with the string object. The `format()` method that we mentioned above is one of them. Some of the commonly used methods are `lower()`, `upper()`, `join()`, `split()`, `find()`, `replace()` etc. Here is a complete list of all the

<https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str>. Let's try some:

```
In [ ]: "Coding is fun!".index("i")
        #That prints out 3, because the location of the first occurrence of the letter
        #Notice how there are actually two i's in the phrase - this method only recog
```

```
In [ ]: "alPacA".lower()
```

```
In [ ]: "alPAca".upper()
```

```
In [ ]: "This will split all words into a list".split()
```

```
In [ ]: 'Happy New Semester'.find('ew') #return index of first occurrence
```

```
In [ ]: 'Happy New Semester'.replace('Happy', 'Brilliant')
```

String Exercises:

```
In [ ]: string1 = "The name of my dog is Charlie!"
        string2 = "My favourite car brand is mercedes benz."
        string3 = "The weather is taking a turn for the worse."
        string4 = "There are many benefits to Python"
        string5 = "    strings are stored as an array of characters    "
```

. For the strings above answer the following:

- Output every second character from string1. Expected output: "Tenm fm o sCale".
- Capitalise the car brand name from string2 and output it. Expected output: "Mercedes Benz".
- Output every fourth character in string3 starting from the third character. Expected output: "ear iaroho".
- Output the length of each string? Expected output: Length: 30,40,43,33,52 respectively.
- Remove the padding (spaces) from string5 and output the new string. What is the length of the shortened string? Expected output: "strings are stored as an array of characters" , length = 44.
- Manipulate the first three strings into a single string with spaces between each sentence and output the result. Expected output: "The name of my dog is Charlie! My favourite car brand is mercedes benz. The weather is taking a turn for the worse."

```
In [ ]:
```

. Given a string of odd length greater than 7, return a new string made of the middle three characters of a given String. Example: "JhonDipPeta" >> Output: Dip

```
In [ ]:
```

. Given two strings, s1 and s2, create a new string by appending s2 in the middle of s1.

Example: `s1 = "Ault" s2 = "Kelly"`, output: `AuKellylt`

In []:

. Given two strings, `s1`, and `s2` return a new string made of the first, middle, and last characters each input string `s1 = "America" s2 = "Japan"` Output `AJrpan`

In []:

2. Conditions:

Python uses boolean logic to evaluate conditions. The boolean values `True` and `False` are returned when an expression is compared or evaluated.

Python supports the usual logical conditions from mathematics:

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

An "if statement" is written by using the `if` keyword. The "and" and "or" boolean operators allow building complex boolean expressions, for example: For example:

In []:

```
name = "John"
age = 23
if name == "John" and age == 23:
    print("Your name is John, and you are also 23 years old.")

if name == "John" or name == "Rick":
    print("Your name is either John or Rick.")
```

In []:

```
password = input('Enter password ')

if password == "myPassword":
    print("Correct password")
else:
    print("Incorrect Password")
```

Nested If: You can have if statements inside if statements, this is called nested if statements.

In []:

```
x = 41

if x > 10:
    print("Above ten,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```

3. Loops

For loops

Iterate over a sequence of numbers using the "range" function. Note that the range function is zero based

```
In [ ]: # Prints out the numbers 0,1,2,3,4
        for x in range(5):
            print(x)
        #try print event numbers in range 5 using range(0,5,2)
```

while loops

In Python, The while loop statement repeatedly executes a code block while a particular condition is true.

In a while-loop, every time the condition is checked at the beginning of the loop, and if it is true, then the loop's body gets executed. When the condition became False, the controller comes out of the block.

```
In [ ]: # Prints out 0,1,2,3,4

        count = 0
        while count < 5:
            print(count)
            count += 1 # This is the same as count = count + 1
```

"break" and "continue" statements

break is used to exit a for loop or a while loop, whereas continue is used to skip the current block, and return to the "for" or "while" statement. A few examples:

The break statement is used inside the loop to exit out of the loop. It is useful when we want to terminate the loop as soon as the condition is fulfilled instead of doing the remaining iterations. It reduces execution time. Whenever the controller encountered a break statement, it comes out of that loop immediately

```
In [ ]: # Prints out 0,1,2,3,4

        count = 0
        while True:
            print(count)
            count += 1
            if count >= 5:
                break

        # Prints out only odd numbers - 1,3,5,7,9
        for x in range(10):
            # Check if x is even
            if x % 2 == 0:
                continue
            print(x)
```

Let's see how to break a for a loop when we found a number greater than 5.

```
In [ ]: for num in range(10):
        if num > 5:
            print("stop processing.")
            break
        print(num)
```

Continue statement in python The continue statement is used to skip the current iteration and continue with the next iteration.

Let's see how to skip a for a loop iteration if the number is 5 and continue executing the body of the loop for other number

```
In [ ]: for num in range(3, 8):
        if num == 5:
            continue
        else:
            print(num)
```

Pass statement in Python

The pass is the keyword In Python, which won't do anything. Sometimes there is a situation in programming where we need to define a syntactically empty block. We can define that block with the pass keyword.

A pass statement is a Python null statement. When the interpreter finds a pass statement in the program, it returns no operation. Nothing happens when the pass statement is executed.

It is useful in a situation where we are implementing new methods or also in exception handling. It plays a role like a placeholder.

Example

```
In [ ]: months = ['January', 'June', 'March', 'April']
        for mon in months:
            pass
        print(months)
```

Can we use "else" clause for loops? When the loop condition of "for" or "while" statement fails then code part in "else" is executed. If a break statement is executed inside the for loop then the "else" part is skipped. Note that the "else" part is executed even if there is a continue statement.

Here are a few examples:

```
In [ ]: # Prints out 0,1,2,3,4 and then it prints "count value reached 5"

        count=0
        while(count<5):
            print(count)
            count +=1
        else:
            print("count value reached %d" %(count))

        # Prints out 1,2,3,4
        for i in range(1, 10):
```

```

if(i%5==0):
    break
print(i)
else:
    print("this is not printed because for loop is terminated because of brea

```

4. Nested loops:

A nested loop is a loop inside the body of the outer loop. The inner or outer loop can be any type, such as a while loop or for loop. For example, the outer for loop can contain a while loop and vice versa.

The outer loop can contain more than one inner loop. There is no limitation on the chaining of loops.

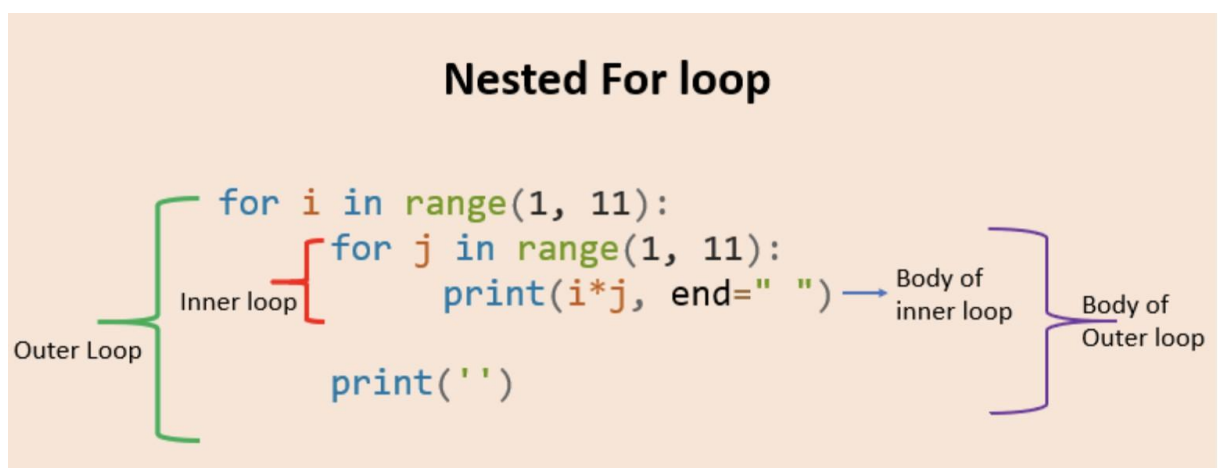
In the nested loop, the number of iterations will be equal to the number of iterations in the outer loop multiplied by the iterations in the inner loop.

In each iteration of the outer loop inner loop execute all its iteration. For each iteration of an outer loop the inner loop re-start and completes its execution before the outer loop can continue to its next iteration.

Nested loops are typically used for working with multidimensional data structures, such as printing two-dimensional arrays, iterating a list that contains a nested list.

A nested loop is a part of a control flow statement that helps you to understand the basics of Python.

- The outer for loop uses the range() function to iterate over the first ten numbers
- The inner for loop will execute ten times for each outer number
- In the body of the inner loop, we will print the multiplication of the outer number and current number
- The inner loop is nothing but a body of an outer loop.



Example: Write a nested for loop program to print multiplication table in Python

```

In [ ]: # outer loop
for i in range(1, 11):
    # nested loop
    # to iterate from 1 to 10
    for j in range(1, 11):

```

```
# print multiplication
print(i * j, end=' ')
print()
```

Another most common use of nested loop is to print various star and number patterns.

Let's see how to use a nested loop to print the following pattern in Python.

```
In [ ]: rows = 5
# outer loop
for i in range(1, rows + 1):
    # inner loop
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

While loop inside a for loop It is very common and helpful to use one type of loop inside another. we can put a while loop inside the for loop.

Assume we wanted to repeat each name from a list five times.

Here we will iterate the list using an outer for loop In each iteration of outer for loop, the inner for loop execute five times to print the current name five times

```
In [ ]: names = ['Kelly', 'Jessa', 'Emma']
# outer loop
for name in names:
    # inner while loop
    count = 0
    while count < 5:
        print(name, end=' ')
        # increment counter
        count = count + 1
    print()
```

Break Nested loop

The break statement is used inside the loop to exit out of the loop. If the break statement is used inside a nested loop (loop inside another loop), it will terminate the innermost loop.

In the following example, we have two loops. The outer for loop iterates the first four numbers using the range() function, and the inner for loop also iterates the first four numbers. If the outer number and a current number of the inner loop are the same, then break the inner (nested) loop.

Example:

```
In [ ]: for i in range(4):
        for j in range(4):
            if j == i:
                break
            print(i, j)
```

Continue Nested loop

The continue statement skip the current iteration and move to the next iteration. In Python, when the continue statement is encountered inside the loop, it skips all the statements

below it and immediately jumps to the next iteration.

In the following example, we have two loops. The outer for loop iterates the first list, and the inner loop also iterates the second list of numbers.

If the outer number and the inner loop's current number are the same, then move to the next iteration of an inner loop.

```
In [ ]: first = [2, 4, 6]
        second = [2, 4, 6]
        for i in first:
            for j in second:
                if i == j:
                    continue
                print(i, '*', j, '= ', i * j)
```

Exercises loops and conditions

. Calculate the sum of first ten numbers (try using while loop)

In []:

. Print a rectangle Pattern with 5 rows and 3 columns of stars

In []:

. Given an input string with the combination of the lower and upper case arrange characters in such a way that all lowercase letters should come first.

In []:

. Write a program to count the total number of digits in a number using a while loop. For example, the number is 75869, so the output should be 5.

Hint 1: This example can be solved with two different ways, try both. Which one you think is better? Discuss with your class mates.

Hint 2: You may need to Set a counter = 0 Run while loop till number != 0 In each iteration of loop Reduce the last digit from the number using floor division (number = number // 10) Increment counter by 1

In []:

In []: *#another sol:*

. Reverse a given integer number, given: 76542, output: 24567

Hint: This example can be solved with two different ways, try both. Which one you think is better? Discuss with your class mates.

In []:

In []: `# another sol`

Challenge:

(1) Find common unique characters between two sentences, while ignoring white space and cases. Example: `s1= "SDPA is fun", s2= 'Alpaca are cute'`, output: `"apu"`

In []:

(2) Write a program to display all prime numbers within a range Note: A Prime Number is a number that cannot be made by multiplying other whole numbers. A prime number is a natural number greater than 1 that is not a product of two smaller natural numbers

Examples: 6 is not a prime number because it can be made by $2 \times 3 = 6$ 37 is a prime number because no other whole numbers multiply together to make it.

In []:

(3) Write a Python program to draw a three stage rocket. Here is mine - you can make yours different!

In [2]:

```

      **
    *  *
  *    *
*=====*
| *    * |
| *  * |
|  **  |
*=====*
| *    * |
| *  * |
|  **  |
*=====*
| *    * |
| *  * |
|  **  |
*=====*

```

More exercises and references:

- <https://pynative.com/python-control-flow-statements/>
- <https://www.codecademy.com/learn/learn-python-3>
- Python style guide <https://www.python.org/dev/peps/pep-0008/>